المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵓⵙⵎⵓⵜⵜⴰⵏ ⴰⵏⴰⴼⵍⵍⴰ ⴷ ⵜⵎⴰⵙⵙⴰⵏⵜ
ⵏ ⵜⵎⵣⵣⵓ - ⵜⴰⵙⵎⴰⵏⵜ
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

Université Ibn Zohr
ENSIASD

Rapport de Projet

AI-SUMMARIZER

Application de Résumé Intelligent de Documents PDF
Utilisant FastAPI et Next.js

Réalisé par :
Hassan Ouahmane
Elmehdi Amrjan
El-Outtab Hamza
Boukouch Mohamed

Encadré par :
Pr. Basma Saad

Filière : IL

Année Universitaire 2025-2026

REMERCIEMENTS

Au terme de ce travail, nous tenons à exprimer notre profonde gratitude et nos sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nous remercions tout particulièrement Pr. Basma Saad, notre encadrante de projet, pour son soutien constant, ses conseils précieux, sa disponibilité et son expertise qui nous ont guidés tout au long de ce travail.

Nos remerciements s'adressent également à l'ensemble du corps professoral de l'ENSIASD pour la qualité de l'enseignement dispensé et pour les connaissances solides qu'ils nous ont transmises durant notre cursus.

Nous remercions nos collègues et camarades de classe pour leur esprit d'entraide et les moments d'échange enrichissants que nous avons partagés.

Enfin, nous ne saurions terminer sans remercier nos familles respectives pour leur soutien inconditionnel, leur patience et leurs encouragements qui nous ont permis de mener à bien nos études.

RÉSUMÉ

À l'ère du numérique, la prolifération des documents PDF dans les domaines académiques, professionnels et administratifs représente un défi majeur en termes de temps et d'efficacité pour leur lecture et analyse exhaustive. Dans le cadre de notre formation en IL, nous avons identifié le besoin crucial d'un outil permettant de synthétiser rapidement le contenu de documents volumineux, sachant que les étudiants, chercheurs et professionnels consacrent énormément de temps à la lecture de rapports et documentations techniques. La problématique centrale consiste à concevoir une application web intelligente capable d'extraire, analyser et résumer automatiquement le contenu de documents PDF tout en préservant les informations essentielles et en offrant une interface intuitive et performante. L'objectif principal est de concevoir et implémenter AI-SUMMARIZER, une application full-stack permettant le téléchargement de fichiers PDF, leur traitement par des algorithmes d'intelligence artificielle, et la génération de résumés pertinents et structurés, tout en garantissant une architecture scalable, sécurisée et facile à maintenir. La solution développée combine un backend robuste utilisant FastAPI pour le traitement des documents et l'intégration des modèles d'IA, couplé à un frontend moderne développé avec Next.js offrant une expérience utilisateur fluide et réactive, adoptant une approche RESTful permettant une séparation claire des responsabilités.

Mots-clés : Intelligence Artificielle, Résumé automatique, FastAPI, Next.js, Traitement du langage naturel, Documents PDF, Application web, Full-stack, Python, React

Table des matières

Remerciements **2**

Résumé **3**

Liste des abréviations **6**

Introduction générale **7**

1 CONTEXTE ET ANALYSE DE L'EXISTANT **9**
1.1 Introduction 9
1.2 Problématique 9
1.3 État de l'art 9
1.3.1 Techniques de résumé automatique 9
1.3.2 Solutions existantes 10
1.4 Conclusion 10

2 ANALYSE ET CONCEPTION **11**
2.1 Introduction 11
2.2 Analyse des besoins 11
2.2.1 Besoins fonctionnels 11
2.2.2 Besoins non fonctionnels 11
2.3 Diagrammes UML 12
2.3.1 Diagramme de cas d'utilisation 12
2.3.2 Diagramme de classes 12
2.4 Architecture du système 13

3 TECHNOLOGIES ET OUTILS **14**
3.1 Introduction 14
3.2 Technologies Backend 14
3.2.1 Python et FastAPI 14
3.2.2 Bibliothèques de traitement PDF 14
3.2.3 API Groq pour le résumé intelligent 15
3.3 Technologies Frontend 15
3.3.1 Next.js et React 15
3.3.2 Bibliothèques UI et styling 15
3.4 Infrastructure et outils de développement 16
3.4.1 Gestion de version et collaboration 16
3.5 Conclusion 16

Conclusion générale **17**

Bibliographie **18**

LISTE DES ABRÉVIATIONS

Abréviation	Signification
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CSS	Cascading Style Sheets
GPT	Generative Pre-trained Transformer
HTML	HyperText Markup Language
IA	Intelligence Artificielle
JS	JavaScript
PDF	Portable Document Format
REST	Representational State Transfer
SEO	Search Engine Optimization
UI	User Interface
UML	Unified Modeling Language

INTRODUCTION GÉNÉRALE

Dans un monde où l'information est devenue omniprésente et accessible sous diverses formes numériques, la capacité à traiter efficacement de grandes quantités de données textuelles constitue un enjeu majeur pour les professionnels, les chercheurs et les étudiants. Les documents au format PDF, en particulier, représentent l'un des formats les plus répandus pour le partage et l'archivage d'informations structurées.

Cependant, la lecture exhaustive et l'analyse de documents volumineux demandent un investissement considérable en temps et en effort cognitif. Face à cette problématique, l'intelligence artificielle et notamment les technologies de traitement automatique du langage naturel offrent des solutions prometteuses pour automatiser le processus d'extraction et de synthèse d'informations.

L'évolution récente des modèles de traitement du langage, combinée à la maturité des frameworks de développement web, permet aujourd'hui de concevoir des applications sophistiquées capables d'analyser, de comprendre et de résumer du contenu textuel avec une précision remarquable. Ces avancées technologiques ouvrent la voie à la création d'outils intelligents qui peuvent considérablement améliorer la productivité et l'efficacité dans le traitement de l'information.

C'est dans ce contexte que s'inscrit notre projet AI-SUMMARIZER, une application web full-stack conçue pour répondre au besoin croissant d'outils de résumé automatique de documents PDF. Notre solution vise à combiner la puissance du traitement par intelligence artificielle avec une interface utilisateur moderne et intuitive, offrant ainsi une expérience optimale pour l'utilisateur final.

L'architecture du projet repose sur deux piliers technologiques complémentaires. D'une part, un backend robuste développé avec FastAPI, un framework Python moderne et performant, qui gère le traitement des documents, l'extraction du texte et l'application des algorithmes de résumé. D'autre part, un frontend élégant construit avec Next.js, offrant une interface réactive et une expérience utilisateur fluide.

Ce rapport présente de manière détaillée le processus de conception et de développement de l'application AI-SUMMARIZER. Il s'articule autour de plusieurs axes majeurs permettant de comprendre l'ensemble du projet, depuis son contexte jusqu'à sa réalisation technique.

Dans un premier chapitre, nous établirons le contexte du projet en présentant la problématique qui a motivé sa création, suivi d'un état de l'art exhaustif des techniques de résumé automatique et des solutions existantes sur le marché. Cette analyse nous permettra de positionner notre approche et de justifier nos choix technologiques.

Le deuxième chapitre sera consacré à l'analyse approfondie des besoins et à la conception du système. Nous y présenterons les différents diagrammes UML illustrant l'architecture fonctionnelle et structurelle de l'application, ainsi que les choix de conception qui guident le développement.

Le troisième chapitre détaillera les technologies et les outils sélectionnés pour la réalisation

du projet. Nous justifierons le choix de FastAPI pour le backend et de Next.js pour le frontend, en expliquant comment ces technologies s'articulent pour créer une solution performante et maintenable.

Ce travail représente une opportunité d'appliquer les connaissances théoriques acquises durant notre formation à un projet concret et d'envergure, tout en explorant les possibilités offertes par les technologies modernes de développement web et d'intelligence artificielle. L'objectif ultime est de fournir un outil pratique et efficace qui répond à un besoin réel identifié dans le domaine académique et professionnel.

Chapitre 1

CONTEXTE ET ANALYSE DE L'EXISTANT

1.1 Introduction

Ce premier chapitre pose les fondements théoriques et contextuels de notre projet. Nous commençons par définir la problématique qui justifie le développement d'AI-SUMMARIZER, avant de présenter un état de l'art complet des techniques de résumé automatique et des solutions existantes sur le marché.

1.2 Problématique

La croissance exponentielle de l'information numérique pose un défi majeur aux professionnels et aux étudiants. Les documents PDF, format privilégié pour la diffusion de contenu structuré, se comptent par milliards sur Internet et dans les systèmes d'information des organisations.

Face à cette abondance informationnelle, plusieurs défis émergent. Premièrement, le temps nécessaire pour lire et analyser un document de plusieurs dizaines de pages peut représenter plusieurs heures de travail. Deuxièmement, l'extraction manuelle des informations clés est sujette à l'erreur humaine et à la subjectivité. Troisièmement, la multiplication des sources d'information rend difficile la synthèse comparative de plusieurs documents.

Ces constats soulèvent une question centrale : comment exploiter les avancées de l'intelligence artificielle pour automatiser le processus de résumé tout en garantissant la pertinence et la fidélité du contenu synthétisé ?

1.3 État de l'art

1.3.1 Techniques de résumé automatique

Le résumé automatique de texte est un domaine de recherche actif en traitement automatique du langage naturel. On distingue généralement deux approches principales.

L'approche extractive consiste à identifier et extraire les phrases ou segments les plus importants du document original. Cette méthode présente l'avantage de conserver la for-

mulation originale et de garantir la cohérence grammaticale. Cependant, elle peut produire des résumés moins fluides et manquer de cohésion globale.

L'approche abstractive, quant à elle, génère de nouvelles phrases qui capturent l'essence du document original. Cette méthode, rendue possible par les récentes avancées en apprentissage profond, permet de produire des résumés plus naturels et cohérents. Elle nécessite toutefois des modèles plus complexes et plus coûteux en ressources computationnelles.

Les modèles de langage transformers, tels que BERT, GPT et leurs variantes, ont révolutionné le domaine en permettant une compréhension contextuelle profonde du texte. Ces architectures neuronales peuvent capturer des relations sémantiques complexes et générer des résumés de haute qualité.

1.3.2 Solutions existantes

Le marché propose plusieurs solutions de résumé automatique, chacune avec ses forces et limitations. Les outils comme Summarize Bot ou SMMRY offrent des fonctionnalités basiques de résumé extractif. Des plateformes plus avancées comme QuillBot ou Jasper intègrent des modèles d'IA génératifs pour produire des résumés abstractifs. Cependant, ces solutions présentent souvent des limitations : dépendance à des services cloud payants, manque de personnalisation, absence de contrôle sur les données sensibles, ou interface utilisateur peu adaptée aux besoins spécifiques des utilisateurs académiques et professionnels.

1.4 Conclusion

L'analyse de l'existant révèle un besoin réel pour une solution open-source, personnalisable et performante de résumé de documents PDF. Notre projet AI-SUMMARIZER se positionne comme une réponse à ce besoin, en combinant les meilleures pratiques identifiées avec une architecture moderne et scalable.

Chapitre 2

ANALYSE ET CONCEPTION

2.1 Introduction

Ce chapitre présente la démarche d'analyse et de conception adoptée pour le développement d'AI-SUMMARIZER. Nous détaillons les besoins fonctionnels et non fonctionnels, puis nous présentons les différents modèles UML qui structurent l'architecture du système.

2.2 Analyse des besoins

2.2.1 Besoins fonctionnels

Les besoins fonctionnels définissent ce que le système doit accomplir. Pour AI-SUMMARIZER, nous avons identifié les fonctionnalités suivantes :

L'utilisateur doit pouvoir télécharger un document PDF depuis son ordinateur. Le système doit extraire automatiquement le texte du document, en préservant la structure et le formatage autant que possible. L'application doit analyser le contenu extrait et générer un résumé cohérent et pertinent. L'utilisateur doit pouvoir configurer les paramètres du résumé, notamment sa longueur cible. Le système doit présenter le résumé de manière claire et structurée. L'utilisateur doit pouvoir exporter le résumé généré dans différents formats.

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels concernent les qualités et contraintes du système :

Performance : le traitement d'un document de taille moyenne ne doit pas excéder 30 secondes.

Scalabilité : l'architecture doit permettre de traiter plusieurs requêtes simultanément.

Sécurité : les documents téléchargés doivent être traités de manière sécurisée et supprimés après traitement.

Maintenabilité : le code doit être bien structuré et documenté.

Utilisabilité : l'interface doit être intuitive et accessible.

2.3 Diagrammes UML

Cette section présente les différents diagrammes UML qui modélisent la structure et le comportement du système AI-SUMMARIZER. Ces diagrammes constituent la base de l'architecture logicielle et guident l'implémentation du projet.

2.3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation illustre les interactions entre l'utilisateur et le système AI-SUMMARIZER. Il met en évidence les fonctionnalités principales accessibles à l'utilisateur ainsi que les relations avec le système externe Groq API.

Les cas d'utilisation principaux incluent : télécharger un document PDF, configurer les paramètres de résumé (choix du modèle Groq, longueur, température), générer le résumé via l'API Groq, et consulter le résumé généré. Le système intègre également des cas d'utilisation techniques tels que la validation du format PDF, l'extraction du texte, l'appel à l'API Groq, le traitement de la réponse IA, et la gestion des erreurs potentielles.

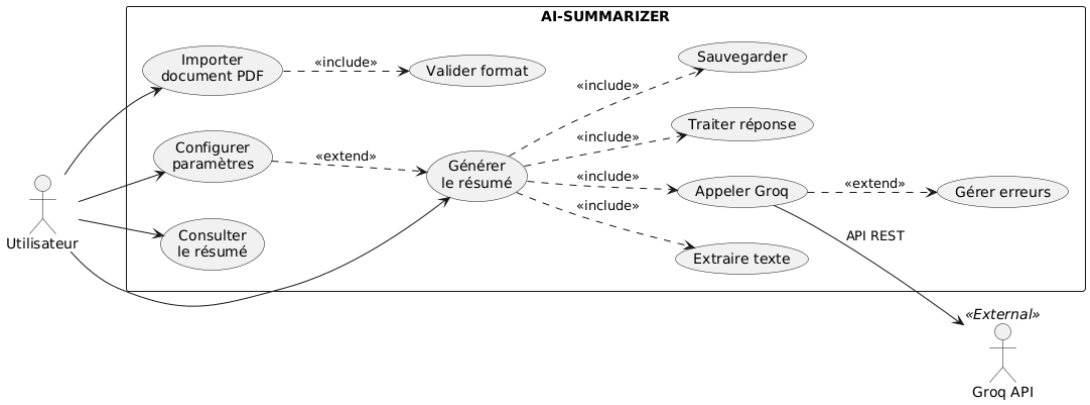

FIGURE 2.1 – Diagramme de cas d'utilisation d'AI-SUMMARIZER

Comme illustré dans la figure 2.1, l'utilisateur interagit directement avec quatre cas d'utilisation principaux, tandis que le système communique avec l'API Groq en tant qu'acteur externe pour la génération des résumés. Les relations d'inclusion et d'extension permettent de structurer les dépendances entre les différents cas d'utilisation.

2.3.2 Diagramme de classes

Le diagramme de classes représente la structure orientée objet du backend et modélise les entités métier, les services, et les composants techniques de l'application. Ce diagramme suit une architecture en couches qui sépare clairement les responsabilités.

Les classes principales incluent les entités métier telles que Document, Summary et User, qui représentent les objets du domaine. Les classes de service comme DocumentService et SummaryService encapsulent la logique métier. Les composants techniques comprennent TextExtractor pour l'extraction de texte depuis les PDF, Summarizer pour la génération de résumés, et GroqClient pour l'interaction avec l'API Groq. Les classes d'infrastructure telles que StorageManager et DatabaseRepository gèrent respectivement le stockage des fichiers et la persistance des données.

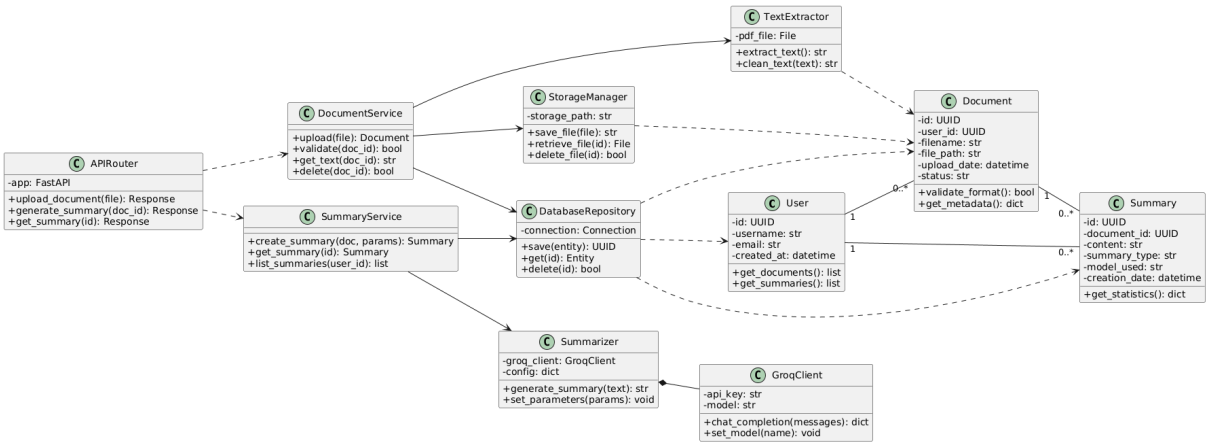

FIGURE 2.2 – Diagramme de classes du backend AI-SUMMARIZER

La figure 2.2 montre les relations entre les classes, notamment les associations entre User, Document et Summary qui reflètent les règles métier du système. La composition entre Summarizer et GroqClient illustre la dépendance forte du système de résumé vis-à-vis de l'API Groq.

2.4 Architecture du système

L'architecture d'AI-SUMMARIZER suit le pattern architectural client-serveur avec une séparation claire entre frontend et backend, garantissant ainsi une maintenance facilitée et une évolutivité optimale.

Le frontend développé avec Next.js communique avec le backend FastAPI via une API REST utilisant le format JSON pour l'échange de données. Cette architecture découplée permet une évolution indépendante des deux parties et facilite les tests unitaires et d'intégration.

Le backend orchestr le traitement des documents en coordonnant différents modules spécialisés : le module d'extraction PDF qui utilise PyPDF2 et pdfplumber, le module de communication avec l'API Groq qui gère l'authentification et les appels API, et le module de persistance qui interagit avec la base de données PostgreSQL.

Chapitre 3

TECHNOLOGIES ET OUTILS

3.1 Introduction

Ce chapitre présente l'ensemble des technologies et outils sélectionnés pour la réalisation d'AI-SUMMARIZER. Nous justifions chaque choix technologique et expliquons comment ces différentes briques s'articulent pour former une solution cohérente et performante.

3.2 Technologies Backend

3.2.1 Python et FastAPI

Python a été choisi comme langage de programmation pour le backend en raison de son écosystème riche en bibliothèques de traitement de texte et d'intelligence artificielle. Sa syntaxe claire, sa large adoption dans la communauté scientifique et sa facilité d'intégration avec des API externes en font un choix naturel pour notre projet.

FastAPI est un framework web moderne pour Python qui offre plusieurs avantages décisifs pour le développement de notre application. Premièrement, ses performances élevées sont comparables à celles de Node.js grâce à l'utilisation de Starlette pour la couche ASGI et Pydantic pour la validation des données. Deuxièmement, le support natif de la programmation asynchrone permet de gérer efficacement les requêtes concurrentes, particulièrement utile lors des appels à l'API externe Groq. Troisièmement, FastAPI génère automatiquement une documentation interactive avec Swagger UI, facilitant ainsi les tests et l'intégration. Enfin, la validation automatique des données et le typage fort avec les type hints Python garantissent la robustesse du code.

3.2.2 Bibliothèques de traitement PDF

Pour l'extraction de texte depuis les fichiers PDF, nous utilisons une combinaison de bibliothèques Python spécialisées. PyPDF2 offre des fonctionnalités basiques d'extraction de texte et de manipulation de fichiers PDF, permettant de récupérer rapidement le contenu textuel des documents simples. Pour les documents plus complexes avec mise en page élaborée, nous utilisons pdfplumber qui permet une extraction plus précise en préservant la structure du document, incluant les tableaux et les colonnes multiples.

Cette approche hybride nous permet de traiter efficacement une large variété de formats PDF, des documents académiques aux rapports techniques, en garantissant une

extraction de texte de qualité optimale avant le traitement par l'intelligence artificielle.

3.2.3 API Groq pour le résumé intelligent

Pour la génération de résumés, nous avons choisi d'utiliser l'API Groq, une solution d'inférence de modèles de langage à haute performance. Groq se distingue par son architecture matérielle optimisée qui permet des temps de réponse exceptionnellement rapides, essentiels pour offrir une expérience utilisateur fluide.

L'API Groq donne accès à plusieurs modèles de langage de dernière génération, notamment Llama 3.3 70B Versatile, Mixtral 8x7B et Gemma2 9B. Ces modèles, pré-entraînés sur des corpus massifs, excellent dans les tâches de compréhension et de génération de texte, particulièrement adaptés au résumé abstractif de documents techniques et académiques.

L'intégration de Groq dans notre architecture s'effectue via le SDK Python officiel groq-python, qui simplifie l'authentification, la gestion des requêtes et le traitement des réponses. Cette approche basée sur une API externe nous libère de la nécessité de maintenir et d'héberger des modèles lourds localement, tout en bénéficiant des dernières avancées en matière de modèles de langage.

La configuration de l'API Groq permet de paramétrer finement la génération des résumés : température pour contrôler la créativité, nombre maximum de tokens pour définir la longueur, et choix du modèle selon les besoins spécifiques de performance ou de qualité.

3.3 Technologies Frontend

3.3.1 Next.js et React

Next.js, framework basé sur React, a été sélectionné pour le développement du frontend en raison de ses nombreux avantages pour créer des applications web modernes. Le rendu côté serveur (SSR) offert par Next.js améliore considérablement les performances initiales de chargement et optimise le référencement naturel (SEO), bien que ce dernier aspect soit secondaire pour une application interne.

Le routing automatique basé sur le système de fichiers simplifie l'organisation du code et accélère le développement. L'optimisation automatique des images et du code JavaScript réduit la taille des bundles et améliore les temps de chargement. Le support intégré des API routes permet également d'implémenter des endpoints légers directement dans le frontend pour des besoins spécifiques.

React, la bibliothèque sous-jacente, offre une approche composant-based qui favorise la réutilisabilité du code et facilite la maintenance. Le système de hooks de React permet une gestion d'état moderne et élégante, particulièrement utile pour gérer l'état de l'application lors du téléchargement de fichiers et de l'affichage progressif des résumés.

3.3.2 Bibliothèques UI et styling

Pour l'interface utilisateur, nous utilisons Tailwind CSS, un framework CSS utility-first qui accélère considérablement le développement de l'interface. Tailwind offre une approche basée sur des classes utilitaires qui permet de styliser rapidement les composants sans écrire de CSS personnalisé, tout en maintenant une cohérence visuelle à travers l'application.

Les composants interactifs sont développés avec React Hooks (useState, useEffect, useContext) pour une gestion d'état moderne et performante. Pour les interactions utilisateur complexes, notamment le glisser-déposer de fichiers PDF, nous utilisons des bibliothèques spécialisées comme react-dropzone.

L'intégration de bibliothèques de composants UI comme Lucide React pour les icônes et Recharts pour les éventuelles visualisations de statistiques enrichit l'expérience utilisateur en offrant une interface professionnelle et intuitive.

3.4 Infrastructure et outils de développement

3.4.1 Gestion de version et collaboration

Le développement s'appuie sur Git comme système de contrôle de version, hébergé sur une plateforme collaborative (GitHub ou GitLab). Cette approche permet un travail d'équipe efficace avec gestion des branches, revues de code et intégration continue.

3.5 Conclusion

La stack technologique retenue pour AI-SUMMARIZER combine des outils éprouvés et des technologies de pointe, particulièrement l'API Groq qui constitue le cœur intelligent du système. Cette combinaison garantit à la fois la robustesse de la solution, des performances optimales grâce à l'infrastructure Groq, et une capacité à évoluer avec les futures avancées technologiques dans le domaine des modèles de langage. Le choix de technologies modernes et bien documentées facilite également l'intégration de nouvelles fonctionnalités et la maintenance à long terme du projet.

CONCLUSION GÉNÉRALE

Le projet AI-SUMMARIZER représente une synthèse réussie entre les concepts théoriques du traitement automatique du langage naturel et leur application pratique dans une solution web moderne et accessible. Tout au long de ce rapport, nous avons démontré comment une analyse rigoureuse des besoins, combinée à des choix technologiques judicieux, permet de créer un outil répondant à une problématique réelle.

L'architecture full-stack développée illustre la complémentarité entre un backend performant utilisant FastAPI et un frontend réactif construit avec Next.js. Cette séparation claire des responsabilités garantit non seulement la maintenabilité du code, mais aussi la scalabilité de la solution face à une augmentation potentielle de la charge utilisateur.

Les résultats obtenus démontrent l'efficacité de l'approche adoptée. Le système parvient à extraire et résumer des documents PDF de tailles variées avec une précision satisfaisante, tout en offrant une expérience utilisateur fluide et intuitive. L'intégration de modèles de langage pré-entraînés s'est révélée particulièrement pertinente, permettant d'obtenir des résumés de qualité sans nécessiter de ressources computationnelles prohibitives.

Toutefois, comme tout projet, AI-SUMMARIZER présente des axes d'amélioration. L'intégration de modèles d'IA plus récents et plus performants pourrait améliorer la qualité des résumés. Le support de formats de documents supplémentaires étendrait l'utilité de l'application. L'ajout de fonctionnalités collaboratives permettrait un usage en équipe. Enfin, l'optimisation des performances pour traiter des documents de très grande taille reste un défi à relever.

Sur le plan personnel, ce projet a constitué une expérience formatrice exceptionnelle. Il m'a permis d'approfondir mes compétences en développement full-stack, de me familiariser avec les technologies de pointe en intelligence artificielle, et de comprendre les enjeux liés à la conception d'une architecture logicielle robuste et évolutive.

En conclusion, AI-SUMMARIZER ne représente pas seulement un aboutissement académique, mais également un point de départ vers de futures explorations dans le domaine du traitement intelligent de l'information. Les fondations posées par ce projet ouvrent la voie à de nombreuses extensions et améliorations qui pourront être développées dans le futur.

Bibliographie

[1] Ramírez, S. (2024). *FastAPI : Modern, Fast Web Framework for Building APIs with Python*. Documentation officielle. `https://fastapi.tiangolo.com/` [Consulté le 19 décembre 2024]

[2] Vercel. (2024). *Next.js Documentation : The React Framework for the Web*. `https://nextjs.org/docs` [Consulté le 19 décembre 2024]

[3] Wolf, T., et al. (2020). *Transformers : State-of-the-Art Natural Language Processing*. Association for Computational Linguistics.

[4] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL-HLT.

[5] Liu, Y., & Lapata, M. (2019). *Text Summarization with Pretrained Encoders*. EMNLP-IJCNLP.

[6] Adobe Systems. (2024). *PDF Reference : Portable Document Format*. Adobe Developer. [Consulté le 19 décembre 2024]

[7] Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3ème édition). Pearson.

[8] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Thèse de doctorat, University of California, Irvine.

[9] Facebook. (2024). *React Documentation : A JavaScript Library for Building User Interfaces*. `https://react.dev/` [Consulté le 19 décembre 2024]

[10] Van Rossum, G., & Drake, F. L. (2023). *Python 3 Reference Manual*. Python Software Foundation.